

legacy-reverse 利用マニュアル

作成 legacy-reverse project

legacy-reverse 利用マニュアル

目次

1	1. skill の配置	3
2	2. hook の登録（必須）	3
3	3. MCP サーバの登録（推奨）	3
4	4. ツール類	3
5	5. 開始	3
6	①②の承認とは何をするとか	4
7	⑤で fail したときの裁定	6
8	WBS サイト	8
9	あなたが直接書いてよいファイル	8

レガシーコード (Fortran / C# など) を Python へ仕様ベースで移植するための、Claude Code skill 群です。関数を 1 つずつ、次の流れで進めます。

①リポジトリ解析 → ①関数仕様書 → ②テスト仕様書 → ③テストコード → ④実装 → ⑤テスト → ⑥完了検証 → ⑦分析・改善

設計の柱は 4 つあります。

1. **クリーンルーム分業** – ②③はレガシー原文を見ない。③は「②+規約」だけ、④は「①+規約」だけを入力に作られる。独立に作ったテストと実装を⑤で突き合わせることで、仕様書の穴が機械的に炙り出される。
2. **ハッシュ連鎖** – ①→②→③の各成果物は上流のハッシュを記録している。上流が改訂されると下流が自動的に「要再生成 (stale 🚩)」になり、伝搬漏れが起きない。
3. **人の承認ゲート** – 仕様の確定・テスト仕様の承認・裁定は必ず人が行う。AI は「仮説 + Yes/No の問い」の形で判断材料を出し、勝手に確定しない。
4. **挙動保存の改善** (㊦) – 移植完了後の高速化・リファクタリングは、③のテスト資産を安全網に「テスト全 pass 維持」を絶対条件として行う。

あなた（人）の仕事は「トリガを引く」「質問に答える」「承認する」「裁定する」の4つです。コードや文書を直接書く必要は基本的にありません（書いてもよい場所は後述）。

图 2

pricing-batch リバースエンジニアリング WBS					目次 進捗サマリ Open ISSUES（要人間判断） 関数一覧（推奨着手順） ⑦ 改善イタレーション										
公開 2026年7月25日															
進捗サマリ															
<table> <tr> <th>①spec</th><th>②test-spec</th><th>③test-code</th><th>④impl</th><th>⑤test</th></tr> <tr> <td>3/3</td><td>3/3</td><td>3/3</td><td>3/3</td><td>3/3</td></tr> </table>						①spec	②test-spec	③test-code	④impl	⑤test	3/3	3/3	3/3	3/3	3/3
①spec	②test-spec	③test-code	④impl	⑤test											
3/3	3/3	3/3	3/3	3/3											
Open ISSUES（要人間判断）															
なし 🚩															
関数一覧（推奨着手順）															
#	関数	依存	①spec	②test-spec	③test-code	④impl	⑤test								
1	get_rate	なし	✅	✅	✅	✅	✅								
2	round_val	なし	✅	✅	✅	✅	✅								
3	calc_price	F-0001, F-0002	✅	✅	✅	✅	✅								
⑦ 改善イタレーション															
ID	分類	目的	対象	状態	達成										
REF-001	refactor	get_rate のフォールバック読みを分離し複雑度を下げる	F-0001	verified	✅										

図 1: WBS（進捗のホーム画面）。進捗サマリ・あなたへの質問（Open ISSUES）・関数一覧・⑦改善イタレーションが 1 画面に集約され、各 から成果物へ移動できる

セットアップ

1 1. skill の配置

対象プロジェクトに skills リポジトリから次をコピーします。

```
cp -r skills/legacy-reverse <project>/claude/skills/  
cp -r skills/legacy-reverse/skills/* <project>/claude/skills/
```

2 2. hook の登録（必須）

legacy-reverse/hooks/settings-example.json の内容を <project>/claude/settings.json に マージします。これは④⑤フェーズ中の tests/ 編集を機械的に拒否する安全装置で、「AI がテストを書き換えて通す」事故をプロンプトではなく仕組みで防ぎます。

3 3. MCP サーバの登録（推奨）

<project>/mcp.json に次を追加すると、台帳操作・テスト実行などの機械処理が 型付きツールになり、許可プロンプトが減って安定します（未登録でも動作は同じ）。

```
{  
  "mcpServers": {  
    "legacy-reverse": {  
      "command": "python",  
      "args": ["<skills リポジトリ>/mcp-servers/legacy-reverse-mcp/server.py"]  
    }  
  }  
}
```

4 4. ツール類

- **Quarto** (HTML/PDF 出力) : 未導入なら quarto-typst-pdf skill の `qtpdf.py install` でポータブル導入（管理者権限不要）
- **Python パッケージ**: `pip install pytest sphinx sphinx-rtd-theme`（⑦では追加で `radon ruff bandit pip-audit`）

5 5. 開始

Claude Code で `/legacy-reverse` を実行すると、セットアップ状態を確認して 次の一手を案内します。新規なら `/legacy-0-analyze` へ。

フェーズ別ガイド – 各フェーズで人がやること

すべてのフェーズは人がスラッシュコマンドで起動します。AIが勝手に次のフェーズへ進むことはありません。

表 5-1

フェーズ	コマンド	人がやること
① 解析	/legacy-0-analyze	レガシーの場所・言語・新パッケージ名を答える。型対応表・規約を一緒に確定
① 仕様書	/legacy-1-spec F-xxxx	レビューして OK を出す（reviewed 化）。●● 項目の質問に答える
② テスト仕様	/legacy-2-testspec F-xxxx	ケース一覧と質問を見て承認（approved 化）。期待値の不明点に答える
③ テストコード	/legacy-3-testcode F-xxxx	基本は見守り（完了時に freeze される）
④ 実装	/legacy-4-impl F-xxxx	基本は見守り。spec-gap ISSUE が来たら①改訂を指示
⑤ テスト	/legacy-5-test F-xxxx	fail 時の裁定（後述）。pass なら WBS で ✓ を確認
⑥ 完了検証	/legacy-6-check	全関数完了後に 1 回。不備リストが出たら該当フェーズへ差し戻し
⑦ 分析・改善	/legacy-7-analyze	代表ワークロードを教える。施策票を承認する

「次にどの関数をやるべきか」は WBS の推奨着手順（依存の葉から）に従います。迷ったら [/legacy-reverse](#) に聞けば [next](#) を提案します。

6 ①②の承認とは何をするのか

AI は承認を求めるとき、**変更点サマリ・Confidence**（● 確認済/● 推測/● 仮定）の内訳・未回答の**質問一覧**をチャットに提示します。あなたは:

- 内容に問題なければ「OK」と返す → AI がフロントマターの status を更新する
- 質問には **Yes / No / 修正内容** で答える（AI は必ず仮説を添えて聞いてくるので、白紙から考える必要はない）
- 答えられない質問は「保留」でよい。ISSUE として残り、WBS の最上部に出続ける

図 6-2

関数仕様書: get_rate

概要

割引区分コードから割引率をテーブル検索して返す。テーブル未ロード時は RATES.DAT から読み込む（フォールバック）。

インタフェース

入力

#	名前	レガシー型	新型	説明	Confidence
1	CODE	CHARACTER*1	str	割引区分コード（1文字、完全一致）	

出力

名前	レガシー型	新型	説明	Confidence
RATE	REAL	float（戻り値テーブル第1要素）	割引率。エラー時 0.0	
IERR	INTEGER	int（戻り値テーブル第2要素）	0=正常 1=コード無し 2=ファイル無し	

グローバル状態

名前	読み/書き	説明	Confidence
/RATTBL/ TCODE(10), TRATE(10), NRATE	読み書き	割引率テーブル（最大10件）。フォールバック時に書き込む	

参照外部ファイル

ファイル	読み/書き	用途	Confidence
RATES.DAT	読み	割引率マスタ。1行 = 区分1文字 + 率5桁 （(A1,F5.3)、例: A0.050）	

呼び出しサブルーチン

名前	func-id	用途
----	---------	----

図 6-1: ①関数仕様書。機械抽出した IO 表と、機能詳細の各項目に Confidence（）とレガシー行番号の根拠が付く。あなたはこれをレビューして OK を出す

ISSUE は必ず「仮説 + Yes/No の問い」の形で来ます。白紙の質問は来ません。

図 6-4

目次

- 概要
- インタフェース
- 機能詳細
- 副作用・例外
- 未確定事項

割引率テーブルの10件打ち切りはバグ互換で維持するか

事象

GETRT のフォールバック読込は RATES.DAT の先頭10件で打ち切り、11件目以降を黙って無視する (legacy/pricing.f:25、配列サイズ10の制約)。マスタが10件を超えた場合、超過分の区分は常に「コード無し(IERR=1)」になる。エラーも警告も出ない。

質問（人への問い）

仮説: これは配列サイズ由来の制約であり意図した仕様ではないが、移植ではバグ互換で10件打ち切りを維持する（現行マスタは10件以内で運用されていると推測。上限拡張は挙動変更になるため⑦以降で検討）。

問い: この理解で正しいですか？（Yes / No / 修正）

回答（人が記入）

- 回答: Yes. バグ互換で10件打ち切りを維持。上限拡張は⑦以降で検討。
- 回答者 / 日付: havealinch / 2026-07-25

反映

反映先	内容
docs/specs/F-0001.md	SPEC-F0001-02 の10件打ち切りを正式仕様として確定

図 6-3: ISSUE の例。AI の仮説に Yes/No/修正 で答えるだけでよい。回答はドメイン知識集に転記され、同じ質問は二度と来ない

7 ⑤で fail したときの裁定

テストが落ちたとき、原因は 3 種類あります。(a) だけは AI が自走し、(b)(c) は あなたの承認が要ります。

表 7-1

分類	意味	あなたの操作
(a) 実装バグ	実装が①を満たしていない	何もしない（AI が④修正→⑤再実行を回す）
(b) テストコードバグ	③が②とズレている	ISSUE の証拠を見て承認 → ③再生成を指示
(c) 仕様が誤り	②①自体が間違い	ISSUE で裁定 → ①改訂を指示（伝搬は自動検知）

(a) のループは 3 回で自動停止し、triage ISSUE が起票されて WBS に ➊ が出ます。このときの再開手順:

- ISSUE を読んで (a)/(b)/(c) を裁定し、回答欄に記入（またはチャットで回答）
- AI が反映 (applied) したら unblock（AI に「F-xxxx をアンブロックして」でよい）
- /legacy-5-test F-xxxx を再トリガ（試行回数は 1 からやり直し）

pricing-batch リバースエンジニアリング WBS ドメイン知識 規約 ⑥完了検証 ⑦分析 新コード詳細(API)							Q
テスト結果: get_rate							目次
サマリ							サマリ
							ケース別結果
							失敗詳細
							試行履歴
仕様ケース数	実装済み	実装率	成功	失敗	スキップ	実行時間	
6	6	100%	6	0	0	0.03s	
ケース別結果							
ケースID	内容	分類	実装	結果	時間	詳細	
F0001-TC-001	ロード済みテーブルからのヒット	正常系	✓	✓	0.0s		
F0001-TC-002	未ロード時のフォールバック読み込	外部ファイルI/O / グローバル状態	✓	✓	0.013s		
F0001-TC-003	10件打ち切り (11件目以降は無視)	境界値	✓	✓	0.015s		
F0001-TC-004	コード不一致	異常系	✓	✓	0.0s		
F0001-TC-005	マスタファイル無し	異常系 / 外部ファイルI/O	✓	✓	0.0s		
F0001-TC-006	空ファイル	境界値	✓	✓	0.001s		
失敗詳細							
なし							
試行履歴							

図 7-1: ⑤テスト結果報告書。実装率（②のケースが③に全部実装されたか）と、ケースごとの内容・分類・結果が 1 枚で分かる。内容列は②の該当ケース定義へのリンク

8 WBS サイト

進捗はすべて HTML サイトで確認します（各フェーズの最後に自動更新されます）。

```
python -m http.server 8765 --directory docs/_site
```

- ・ **トップ** = **WBS**: 進捗サマリ、Open ISSUE（あなたへの質問が常に最上部）、関数一覧（各 ☒ から成果物へリンク）、⑦改善イタレーションの達成状況
- ・ ナビバー: ドメイン知識 / 規約 / ⑥完了検証 / ⑦分析 / 新コード詳細(API)
- ・ API ページ (Sphinx) : 関数一覧 → 個別ページ。Spec: 行で仕様書へ、トレース対応表で func-id ↔ 新関数 ↔ レガシー名 ↔ 仕様書が相互に辿れる

図 8-2

The screenshot shows a web browser displaying the Sphinx-generated API documentation for 'pricing-batch'. The page has a dark blue sidebar on the left with a search bar and a list of functions: `pricing.rates.get_rate`, `pricing.rounding.round_val`, and `pricing.calc.calc_price`. The main content area is white and features the title 'pricing-batch 新コード詳細仕様'. Below the title, there is a note about docstring generation and a 'Spec:' link. A section titled '関数一覧' (Function List) contains a table with three rows, each showing a function name and its description. Below this is a 'トレース対応表' (Traceability Table) with four columns: 'func-id', '新関数' (New Function), 'レガシー' (Legacy), and '関数仕様書' (Function Specification). The table lists three functions with their IDs, new function names, legacy names, and links to their specifications. At the bottom of the page, there is a 'Next' button and a copyright notice.

func-id	新関数	レガシー	関数仕様書
F-0001	<code>pricing.rates.get_rate()</code>	GETRT	仕様書
F-0002	<code>pricing.rounding.round_val()</code>	RNDVAL	仕様書
F-0003	<code>pricing.calc.calc_price()</code>	CALCPR	仕様書

図 8-1: 新コード詳細(API)。docstring から自動生成され、関数一覧→個別ページ、トレース対応表から仕様書へ渡れる

9 あなたが直接書いてよいファイル

表 9-1

ファイル	手編集	説明
docs/domain-knowledge.md • docs/conventions.md	○	あなたが著者。業務知識・規約はここへ
ISSUE の「回答（人が記入）」欄	○	じっくり書きたい裁定はファイルで
docs/specs/ （仕様書）	△	編集可。ハッシュ連鎖が下流を stale にして再確認が走る（設計どおり）
WBS・テスト結果・完了検証レポート	✕	自動生成。次の再生成で消える

ファイルに直接書いた内容は、次にどのフェーズを起動したときに AI が自動で拾います（全 skill は起動時に「回答済み ISSUE・規約変更」をスキャンしてから本処理に入ります）。チャットで「DK に追加して: ○○」と言うだけでも構いません。

⑦ 分析・改善の回し方

⑥が pass した後、`/legacy-7-analyze` で開始します。1 施策 = 1 票 = 1 イタレーションの DevOps ループです。

1. **計測・走査**: AI が `profile / radon / bandit` を実行し、候補を `analysis.md` に列挙。このとき**代表ワークロード（実データ規模の入力）を聞かれるので用意する**。単体テスト負荷だけではホットスポットを断定しません
2. **項目確定**: 着手する候補が施策票（`docs/improvements/OPT-001.md` など）に昇格。票には目的・**検証可能な成功基準（baseline→目標）**・改善仕様が書かれる。**あなたはこの票を承認する**
3. **イタレーション**: AI が適用（1 施策=1 コミット）→ テスト全 pass 確認 → 再計測 → 票に実測と判定を記録。全基準達成なら achieved 、未達なら巻き戻して振り返り
4. WBS の「⑦改善イタレーション」表で全施策の達成状況（//-）がトレースできる

Rust(PyO3) 化のような大きな施策も、AI が 4 段階基準（CPU バウンドか → NumPy で足りないか → 純粋関数か → 保守コスト明記）で根拠付きの提案を出すので、票の承認で判断してください。

図 9-2

get_rate のフォールバック読込を分離し複雑度を下げる

目的（Plan）

`get_rate` はプロジェクト唯一の複雑度B評価（CC=8）。マスタ読込と検索という2つの責務が同居しているため、将来マスタ形式が変わったときの修正範囲を読込側に閉じ込められるよう、読込を `private` 関数 `_load_rate_master()` に分離して両関数をA評価にする。

成功基準（検証可能な形で定義する）

#	指標	baseline	目標	実測(after)	判定
1	radon CC: <code>get_rate</code>	B(8)	A(5)以下	A(5)	
2	radon CC: 分離後の全関数	—	全てA	<code>get_rate</code> A(5), <code>_load_rate_master</code> A(5)	
3	radon MI: <code>rates.py</code>	A(83.47)	A維持	A(80.02)	
4	⑤テスト (15ケース)	全pass	全pass維持	15 passed	

改善仕様（Do の設計）

- `rates.py` に `_load_rate_master() -> int` を新設（0=成功/読込済み扱い、2=ファイル無し）。`RATES.DAT` のオープン・（A1,F5.3）パース・10件打ち切り・STATE への格納をここへ移す
- `get_rate` は「未ロードなら `_load_rate_master()`」、エラー2なら（0.0, 2）、あとは検索」に縮む
- **挙動保存の根拠**: 外部から見た入出力（引数・戻り値・STATE の事後状態・ファイルアクセス）は一切変えない。関数境界の移動のみ。テスト15ケース（②F0001-TC-001～006を含む）が判定する
- `_` 付き `private` のため Sphinx の公開APIには現れない（ドキュメント影響なし）

検証記録（Check）

- 2026-07-25 適用。`_load_rate_master()` を新設し読込処理を移動、`get_rate` は判定+検索のみに
- `pytest tests -q` → **15 passed**（②由来の特性テスト全維持。挙動保存を確認）
- `radon cc: get_rate` B(8) → **A(5)**、`_load_rate_master` A(5)。B評価消滅
- `radon mi: A`(83.47) → A(80.02)（Aランク維持）

振り返り（Act）

目次

目的（Plan）

成功基準（検証可能な形で定義する）

改善仕様（Do の設計）

検証記録（Check）

振り返り（Act）

図 9-1: ⑦施策票の例 (REF-001)。目的→成功基準 (baseline→目標→実測→判定) →検証記録→振り返りが 1 枚に揃い、
目的が達成されたかが後から機械的に追える

トラブルシューティング

表 9-2

症状	意味	対処
WBS に  ISSUE-xxx	④⑤ループが上限到達、裁定待ち	ISSUE に回答 → unblock → ⑤再トリガ
WBS に stale 	上流 (①) が改訂され②が古い	<code>/legacy-2-testspec</code> で再生成→再承認
WBS に  改変	freeze 後にテストコードが変わった	意図した変更なら③で再 freeze。 心当たりがなければ調査
tests/ 編集が拒否された	hook が正常動作している	テスト側の疑義は ISSUE 経由で (正規経路)
再レンダリングが失敗する	配信サーバが <code>_site</code> をロック	サーバの <code>cwd</code> を <code>_site</code> の外にして <code>--directory</code> 指定

PDF 出力

種別ごとの合本 PDF は次で生成します（フォールバック含め自動処理）。

```
python <LR>/scripts/pdf_book.py specs      --root . --output pdf/関数仕様書.pdf --title 関数仕様書
python <LR>/scripts/pdf_book.py test-specs  --root . --output pdf/テスト仕様書.pdf --title テスト仕様書
python <LR>/scripts/pdf_book.py test-results --root . --output pdf/テスト結果報告書.pdf --title テスト結果報告書
```

生成後は `qtpdf.py check <pdf>` で豆腐・はみ出しの機械チェックができます。

付録: 成果物とデータの全体像

```
docs/
index.qmd      WBS (自動生成・手編集禁止)
conventions.md プロジェクト規約 (⑩で確定・人が編集可)
domain-knowledge.md  裁定の蓄積 (人が編集可)
specs/F-xxxx.md    ① 関数仕様書 (skeleton→draft→reviewed)
test-specs/F-xxxx.md ② テスト仕様書 (generated→approved)
test-results/F-xxxx_日時.md ⑤ 結果報告書 (毎回新規・自動生成)
issues/ISSUE-xxx.md  質問と裁定 (回答欄はあなたが記入可)
improvements/XXX-nnn.md ⑦ 施策票 (proposed→approved→applied→verified)
completion-check.md  ⑥ 完了検証 (自動生成)
data/
functions.json  ⑩の解析結果 (正データ)
ledger.json     ハッシュ・ブロック状態 (スクリプト専用)
src/ , tests/   ④実装 / ⑤テスト (④⑤中は hook が tests/ を保護)
```

ステータスの意味: =完了 / =作業中 (draft 等) / =未着手 / =裁定待ち / =要再確認